



Ingenieria Informatica

Procesadores de Lenguaje 25/26

Grupo 81

Practica Final - Entrega Final

«Traductor de un subconjunto de Lisp a
Forth»

Equipo 121:

Denis Loren Moldovan — 100522240

Jorge Adrian Saghin Dudulea — 100522257

Profesor

MARIA PAZ SESMERO LORENTE

Tabla de contenidos

1. Primera aproximación	2
1.1. Estructura de control IF	2
1.2. Variables locales	2
2. Segunda aproximación	3
2.1. Variables locales	3
2.2. Bucle for	3
2.3. switch / case	3
2.4. Funciones	3
2.5. Vectores	4
3. Tercera aproximación	5
3.1. Nuevos tokens	5
3.2. Variables: setq y setf	5
3.3. Print y princ	5
3.4. Defun main y llamada a main	5
3.5. Bucle: loop while do	5
3.6. Condicional: if	5
3.7. Operadores aritméticos y relacionales	6
4. Cambios realizados como parte de la entrega final	7
5. Pruebas realizadas	8
6. Declaración de uso de IA	9

1. Primera aproximación

1.1. Estructura de control IF

En este punto se pedía implementar la traducción de la estructura if/else, donde las reglas de traducción son:

- `if (<expr>) { <codigo> }`
 - `(if <expr> <codigo>)`
- `if (<expr>) { <codigo1> } else { <codigo2> }`
 - `(if <expr> <codigo1> <codigo2>)`

El uso de `progn` permite agrupar múltiples sentencias en cada rama, tal como exige Lisp. Los bloques `{ y }` son obligatorios, lo que resuelve el conflicto `shift/reduce` que menciona el enunciado. La regla `control_if` está integrada dentro de cuerpo, por lo que puede aparecer correctamente dentro del `main`.

1.2. Variables locales

Solamente se han puesto las declaraciones necesarias para llevar a cabo esta parte. El objetivo es, a diferencia de las variables globales, es concatenar el nombre de la función, con un guión bajo, al nombre de la función (en vez de `-` ya que este sería ambiguo), con el objetivo de poder ser llamadas solamente en su respectivo scope.

Para saber si una variable usada dentro de una función es local o global, se va a mantener una tabla de variables locales donde se inserten todas las declaradas dentro del cuerpo de la función. Si una variable no aparece en esa tabla, es global y no se concatena.

2. Segunda aproximación

2.1. Variables locales

Para implementar las variables locales, hemos añadido funciones y variables globales de `c` para poder detectar si estamos en una función y almacenar las variables locales. La variable local `en_funcion` indica 0 si estamos fuera de una función y 1 si estamos dentro. Para implementarlo, lo que hemos hecho ha sido añadir acciones semánticas que setean el valor de `en_funcion` a 1 después de que se hayan parseado los tokens de “)” que contienen la lista de argumentos de las funciones. Y al acabar de parsear el cuerpo y el “}”, le asignamos el valor 0 a la función. También hemos implementado funciones que borran la lista de variables locales.

2.2. Bucle for

El bucle `for` de C se traduce descomponiéndolo en dos partes separadas de Common Lisp: la inicialización como una asignación independiente (`setq/setf`), y el resto como un `loop while` equivalente, donde el incremento/decremento se coloca al final del cuerpo del bucle.

La gramática acepta como inicialización tanto una asignación a variable ya existente como una declaración con tipo (`int i = 0`), en cuyo caso se registra la variable como local y se le añade el prefijo del nombre de la función para evitar colisiones con variables globales. Como operación de iteración solo se admiten `inc(var)` y `dec(var)`, que se traducen a (`setf var (+ var 1)`) y (`setf var (- var 1)`) respectivamente.

Por ejemplo, `for (int i = 0; i < 10; inc(i)) { x = x + i; }` se traduce a:

```
(setq main_i 0)
(loop while (< main_i 10) do
  (setf main_x (+ main_x main_i))
  (setf main_i (+ main_i 1)))
```

2.3. switch / case

El `switch` statement no tiene ninguna implementación complicada. Al igual que el resto de la gramática, simplemente hemos añadido el token para el `SWITCH`, y en cada caso se puede insertar el cuerpo completo, si al final encuentra el `break`.

Además, estos se pueden encadenar dentro del cuerpo, para añadir más `switches` en niveles superiores.

2.4. Funciones

Las funciones de C se traducen a `defun` de Common Lisp. La gramática reconoce la cabecera con su lista de argumentos y el cuerpo con sus sentencias, generando directamente la forma (`defun nombre (args) cuerpo`).

Para gestionar las variables locales, cada vez que se entra en una función se limpia la tabla de variables locales (`vars_locales`). Toda variable declarada dentro de la función se registra en esa tabla y se le añade el prefijo `nombre_funcion` en el código generado,

evitando así colisiones con variables globales del mismo nombre. Cuando se referencia un identificador, se comprueba si está en la tabla local y se aplica el prefijo correspondiente.

Los argumentos formales, en cambio, no se prefijan: se usan tal cual en la definición del defun, ya que Lisp los trata como variables locales al ámbito de la función de forma nativa.

El return se traduce a (return-from nombre_funcion valor). Si el return es la última sentencia y no hay código posterior ni está dentro de una rama (nivel_rama == 0), se omite el return-from y se deja la expresión directamente, aprovechando que Lisp devuelve implícitamente el valor de la última forma evaluada.

Por ejemplo:

```
int suma(int a, int b) {  
    int r = a + b;  
    return r;  
}
```

Se traduce a:

```
(defun suma (a b)  
  (setq suma_r (+ a b))  
  suma_r)
```

2.5. Vectores

Los vectores se pueden declarar y definir globalmente, usarse como operandos, y acceder y modificar dentro del cuerpo de las funciones.

Al igual que con las variables globales, los vectores también usan los mismo métodos para comprobar que esté dentro del scope de cada función.

3. Tercera aproximación

3.1. Nuevos tokens

Se han añadido los siguientes tokens respecto a la versión inicial: OR, NEQ ($\neq$), LTEQ ($\leq$), GTEQ ($\geq$), MOD y NOT. Los operadores de dos caracteres se detectan mediante el mecanismo de *expandable operators*, que lee un segundo carácter al encontrar uno de los caracteres iniciales.

3.2. Variables: `setq` y `setf`

(`setq var expr`) declara una variable global en Forth e inicializa su valor. Hay dos variantes: si el valor inicial es un literal numérico emite `variable <nombre> <valor> <nombre> !`; si es una expresión emite `variable <nombre>`, el código de la expresión, y luego `<nombre> !`. (`setf var expr`) asigna a una variable ya existente, emitiendo solo `<nombre> !` tras la expresión:

```
(setq x 5)           → variable x 5 x !
(setf x (+ x 1))     → x @ 1 + x !
```

3.3. Print y princ

- (`print "texto"`) emite `.\ " texto\" cr` (con salto de línea).
- (`princ expr`) emite `.` tras evaluar la expresión (sin salto de línea).
- (`princ "texto"`) emite `.\ " texto\"` (sin salto de línea).

3.4. Defun main y llamada a main

La definición de la función principal se traduce a una palabra Forth. La acción semántica emite `: main` al reducir la cabecera y `;` al cerrar el cuerpo. La llamada (`main`) emite simplemente `main`:

```
(defun main () <cuerpo>) → : main <cuerpo> ;
```

3.5. Bucle: `loop while do`

El bucle (`loop while <cond> do <cuerpo>`) sigue el patrón `BEGIN ... WHILE ... REPEAT` de Forth. Las tres acciones semánticas emiten `BEGIN` antes de la condición, `WHILE` tras evaluarla y `REPEAT` al cerrar:

```
(loop while (< i 10) do (setf i (+ i 1))) → BEGIN i @ 10 < WHILE i @ 1
+ i ! REPEAT
```

3.6. Condicional: `if`

Las acciones emiten `IF`, `ELSE` y `THEN` en los momentos adecuados de la reducción:

```
(if (< x 0) (setf x 0))           → x @ 0 < IF 0 x ! THEN
(if (< x 0) (setf x 0) (setf x 1)) → x @ 0 < IF 0 x ! ELSE 1 x ! THEN
```

3.7. Operadores aritméticos y relacionales

Se han añadido las siguientes reglas a la producción *expression*, todas con traducción directa en notación postfija:

Lisp	Forth	Lisp	Forth	Lisp	Forth
(+ a b)	a b +	(and a b)	a b and	(= a b)	a b =
(* a b)	a b *	(or a b)	a b or	(/= a b)	a b = 0=
(/ a b)	a b /	(not a)	a 0=	(< a b)	a b <
(- a b)	a b -	(<= a b)	a b <=	(> a b)	a b >
(- a)	a negate	(mod a b)	a b mod	(>= a b)	a b >=

La desigualdad /= se traduce como = 0= y not como 0=, ya que Forth no tiene operadores dedicados para estos casos.

4. Cambios realizados como parte de la entrega final

Al compilar con bison el back.y, hemos obtenido un conflicto shift-reduce en las reglas siguientes:

```
Example: '(' SETQ IDENTIF number • ')'
Shift derivation
  expression1
↳ 7: '(' SETQ IDENTIF number • ')'
Reduce derivation
  expression1
↳ 8: '(' SETQ IDENTIF expression      ')'
                                ↳ 24: operand
                                ↳ 41: number •
```

Para arreglar este conflicto, solo tuvimos que eliminar la regla '(' SETQ IDENTIF number • ')' para evitar redundancias.

Otra cosa que hemos cambiado en esta sesión ha sido el volcado de la traducción en el axioma. Lo hemos quitado de las acciones semánticas del no terminal del axioma y hemos incluido los printf en las acciones semánticas de las reglas de declaraciones_funciones y de funcion_main.

5. Pruebas realizadas

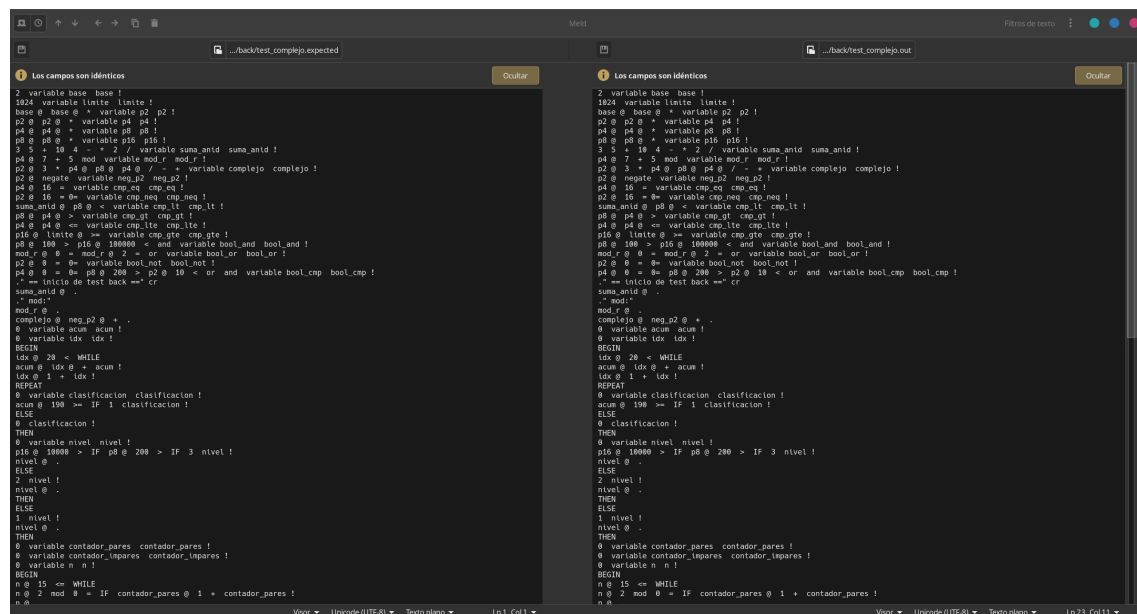
Para las pruebas realizadas, hemos usado un set de tests generados por Claude, a partir del código final, tanto del backend como del frontend. Hemos tomado en cuenta que este consistiría básicamente por snippets de código reales de C y Lisp, por lo que, tras revisar la salida y limpiarla, la hemos usado para comprobar el correcto funcionamiento de este.

Para la ejecución de tests, hemos seguido la siguiente dinámica. Para el front-end, hemos usado el siguiente mandato:

```
$ cat test_complejo.c | ../../trad >test_complejo.out | meld --diff
test_complejo.expected test_complejo.out
```

Y para el backend, este otro:

```
$ cat test_complejo.lisp | ../../back >test_complejo.out | meld --diff
test_complejo.expected test_complejo.out
```



6. Declaración de uso de IA

Hemos hecho uso de la inteligencia artificial generativa para la generación de los casos de prueba de las gramáticas ya desarrolladas, para evitar perder el tiempo escribiendo snippets genéricos de código, que puedan llegar a estar mal, y alimentarlos directamente a nuestros programas.

Para esta tarea, hemos usado la versión gratuita de Claude, alimentandolo como contexto nuestra gramática, y revisando completamente la salida completa para evitar errores.